

Distributed Task Queue System

A fault-tolerant, horizontally scalable task queue built with Go, Redis, PostgreSQL, React, and WebSockets.

Repository	1CellDotAi/distributedTaskQueue
Primary language	Go 1.25 (backend) + TypeScript/React (dashboard)
Key infrastructure	Redis, PostgreSQL, Docker Compose
Document	Project guide – overview, modules, API, design & concepts

Table of Contents

- 1. Project Overview
- 2. How to Run the Project
- 3. Repository Layout & Modules
- 4. REST + WebSocket API Reference
- 5. Core Concepts Used
- 6. High-Level System Design
- 7. Low-Level Design
- 8. Configuration Reference
- 9. Testing & Validation

1. Project Overview

The Distributed Task Queue is a production-grade background-job system. Clients submit tasks via an HTTP API; the system persists them in PostgreSQL, schedules them through Redis, executes them on a pool of horizontally scalable workers, and streams live lifecycle events to a React dashboard over WebSockets.

Key features

- **Priority scheduling** using Redis sorted sets (lower priority value = more urgent).
- **Exponential backoff retries** with full jitter and a separate delayed: * ZSET that a coordinator promotes back to the ready queue.
- **Heartbeat-based failure detection**: workers refresh a Redis TTL key every few seconds; if the key disappears, the coordinator reclaims their in-flight tasks (target recovery < 5s).
- **At-least-once delivery + idempotency** via an atomic Lua ZPOPMIN + HSET lease, with explicit ACK / NACK and a per-worker in-flight hash.
- **Dead-letter queue** for tasks that exceed max_attempts, with redrive from the dashboard.
- **Zero data loss**: tasks are persisted to PostgreSQL *before* they are enqueued in Redis; the coordinator reconciles orphaned running rows on worker death.
- **Live dashboard** (React + TanStack Query) subscribed to a WebSocket fed by Redis pub/sub.

2. How to Run the Project

2.1 Docker Compose (recommended)

This spins up PostgreSQL, Redis, the API server, three workers, the scheduler/coordinator, and the dashboard in one command.

```
# Build and start the full stack
docker compose up --build

# Dashboard:  http://localhost:5173
# API:        http://localhost:8080/api
```

Submit a sample task once the stack is up:

```
curl -X POST http://localhost:8080/api/tasks \
  -H 'Content-Type: application/json' \
  -d '{"type":"echo","payload":{"hello":"world"},"priority":1,"max_attempts":3}'
```

2.2 Running locally (without containers)

Bring up PostgreSQL and Redis any way you like, then run each Go binary in its own terminal.

```
export DATABASE_URL='host=localhost port=5432 user=postgres dbname=taskq sslmode=disable'
export PGPASSWORD='postgres'
export REDIS_URL='redis://localhost:6379/0'

go run ./cmd/api           # REST + WebSocket server (:8080)
go run ./cmd/worker        # Task executor (heartbeats, leases)
go run ./cmd/scheduler     # Coordinator (delayed promoter + reclamer)

# Dashboard
```

```
cd web && npm install && npm run dev    # http://localhost:5173
```

2.3 Database migrations

Schema lives in migrations/0001_init.sql. The Docker Compose setup applies it automatically. For local runs, apply it manually with any Postgres client (for example psql -f migrations/0001_init.sql).

3. Repository Layout & Modules

```
cmd/
  api/      REST + WebSocket server entry point
  worker/   Task-executing worker (handler registry)
  scheduler/ Coordinator (delayed promoter + dead-worker reclaimer)
  loadtest/ Submits N tasks and reports failure rate
internal/
  config/   Env-var based configuration loader
  store/    PostgreSQL repository (pgx)
  queue/    Redis abstraction (priority, delayed, in-flight, DLQ)
  task/     Task model, handler interface, sample handlers
  scheduler/ Worker runtime + Coordinator implementation
  api/      HTTP handlers (chi router)
  ws/       WebSocket hub fed by Redis pub/sub
migrations/ SQL schema
web/        React + Vite dashboard
scripts/    loadtest.sh, kill-worker.sh
```

3.1 cmd/ – binaries

Binary	Role
cmd/api	Boots the chi HTTP router, mounts REST handlers and the /ws WebSocket endpoint. Connects to Postgr
cmd/worker	Registers handlers (echo, sleep, flaky, email), claims a worker ID, starts the heartbeat loop and a pool of
cmd/scheduler	Runs the Coordinator: periodically promotes due tasks from delayed:{type} into queue:{type}, and detects
cmd/loadtest	Stress utility: submits N tasks at C concurrency, polls /api/stats until drained, prints failure rate.

3.2 internal/ – packages

Package	Responsibility
config	Reads environment variables and returns a Config struct with sensible defaults (heartbeat intervals, retry dela
store	PostgreSQL persistence using pgx. CRUD for tasks, an immutable task_events audit table, and worker regist
queue	All Redis operations: Enqueue/Dequeue via Lua, lease/Ack/Nack, promotion of delayed entries, DLQ push/lis
task	Domain model (Task, Status, Event) plus the Handler interface, HandlerFunc adapter, Registry and built-in ha
scheduler	Worker runtime (the loop that pulls, executes, and acks/nacks tasks with backoff) and Coordinator (delayed p
api	chi handlers for the REST endpoints and the /ws upgrade.
ws	WebSocket Hub: maintains client connections, fans out events received on the Redis events:tasks channel.

3.3 web/ – dashboard

React 18 + Vite + TanStack Query single-page app served from web/src. It calls the REST API for snapshots and subscribes to /ws for live lifecycle updates (queued, running, succeeded, failed,

retrying, dead).

4. REST + WebSocket API Reference

4.1 Endpoints

Method & Path	Description
POST /api/tasks	Submit a new task.
GET /api/tasks?status=&type=&limit=	List tasks, optionally filtered.
GET /api/tasks/{id}	Fetch a single task by id.
GET /api/tasks/{id}/events	Full audit trail of state changes for a task.
POST /api/tasks/{id}/cancel	Cancel a queued or retrying task.
GET /api/workers	List workers and their last heartbeat.
GET /api/dlq	List dead-letter records.
POST /api/dlq/{id}/redrive	Re-enqueue a dead task back into its ready queue.
GET /api/stats	Aggregate counts by status + queue depths.
GET /healthz	Liveness probe.
GET /readyz	Readiness probe (Postgres + Redis reachable).
GET /ws	WebSocket; streams task.Event JSON payloads.

4.2 Submit a task – syntax

Request body:

```
POST /api/tasks
Content-Type: application/json
```

```
{
  "type":          "echo",           // registered handler type
  "payload":       { "hello":"world" }, // arbitrary JSON
  "priority":      1,                // lower = more urgent
  "max_attempts":  3                  // total tries before DLQ
}
```

Response (201 Created):

```
{
  "id": "8c0e3f8a-...-uuid",
  "type": "echo",
  "status": "queued",
  "priority": 1,
  "attempts": 0,
  "max_attempts": 3,
  "available_at": "2026-06-02T10:00:00Z",
  "created_at": "2026-06-02T10:00:00Z",
  "updated_at": "2026-06-02T10:00:00Z"
}
```

4.3 Lifecycle of a task

- **queued** – persisted in Postgres and ZADD-ed into queue : {type}.

- **running** – a worker leased it via Lua ZPOPMIN + HSET into `inflight:{worker}`.
- **succeeded** – handler returned without error; row updated, in-flight entry HDEL-ed.
- **retrying** – handler errored and `attempts < max_attempts`; task placed into `delayed:{type}` with backoff.
- **failed/dead** – attempts exhausted; pushed onto `dlq:{type}`.
- **canceled** – removed by the cancel endpoint while queued/retrying.

4.4 WebSocket events

Connect to GET /ws. Each message is a JSON `task.Event`:

```
{
  "task_id":    "8c0e3f8a-...",
  "type":       "echo",
  "status":     "succeeded",
  "worker_id":  "worker-7",
  "message":    "",
  "timestamp":  "2026-06-02T10:00:01.123Z"
}
```

4.5 Other useful calls

```
# List dead-letter records
curl http://localhost:8080/api/dlq

# Redrive a dead task
curl -X POST http://localhost:8080/api/dlq/<id>/redrive

# Aggregate stats
curl http://localhost:8080/api/stats

# Cancel a task
curl -X POST http://localhost:8080/api/tasks/<id>/cancel
```

5. Core Concepts Used

5.1 At-least-once delivery with leases

When a worker dequeues a task it executes a single Lua script that atomically ZPOPMINs from `queue:{type}` and HSETs the envelope into `inflight:{worker}`. Until the worker explicitly ACKs (HDEL on success) or NACKs (re-enqueue with delay), the task remains owned by that worker. If the worker dies, the coordinator inherits the in-flight hash and re-enqueues every entry, guaranteeing the task runs at least once.

5.2 Priority scheduling via sorted sets

Tasks are placed into `queue:{type}` with score `priority * 1e13 + enqueueMs`. ZPOPMIN therefore returns the highest-priority, oldest task first, giving a stable FIFO ordering within a priority.

5.3 Exponential backoff with jitter

On NACK the worker computes `delay = min(RETRY_MAX_DELAY, RETRY_BASE_DELAY * 2^(attempt-1))` and applies full jitter (uniform in `[0, delay]`). The envelope is ZADD-ed to

delayed:{type} with score = now + delay. A coordinator tick promotes due entries back into the ready ZSET.

5.4 Heartbeats and failure detection

Each worker periodically SET hb:{worker} EX HEARTBEAT_TTL. The coordinator scans the worker registry and treats a missing hb:* key as death. It then reads inflight:{worker}, requeues each task and deletes the hash, achieving recovery in < 5 seconds with default settings.

5.5 Idempotency & zero data loss

Postgres is the source of truth. A task row is inserted before it is ZADD-ed to Redis, and every status transition writes a row to task_events. On coordinator reclaim, orphaned running rows are reconciled back to queued/retrying before redelivery, so duplicates are handler-detectable and no work is silently lost.

5.6 Dead-letter queue (DLQ)

When attempts >= max_attempts the envelope is LPUSH-ed into dlq:{type} and the task row is marked dead. Operators can list and redrive DLQ items from the dashboard or via POST /api/dlq/{id}/redrive.

5.7 Event streaming via pub/sub

Every state change is JSON-encoded and PUBLISH-ed to the Redis events:tasks channel. The WebSocket Hub SUBSCRIBE-s once and fans messages out to every connected dashboard client.

5.8 Handler registry & pluggability

task.Registry maps a string type to a Handler. Workers only consume types they have registered (or all if TASK_TYPES is empty), enabling fleet specialisation. Built-in handlers are echo, sleep, flaky, email; new ones plug in via registry.Register("yourtype", handler).

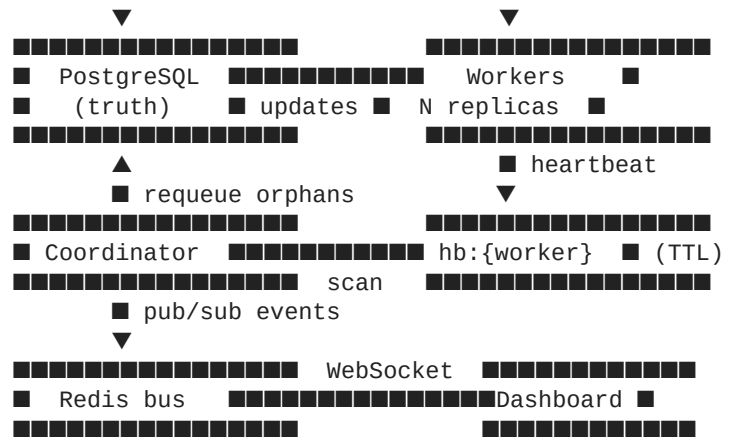
6. High-Level System Design

The system is composed of four logical planes that scale independently:

- **Submission plane** – stateless API replicas behind a load balancer; persist to Postgres, enqueue to Redis.
- **Execution plane** – N stateless workers consuming from priority ZSETs; horizontally scalable.
- **Coordination plane** – a singleton (or leader-elected) scheduler promoting delayed tasks and reclaiming dead workers.
- **Observability plane** – Redis pub/sub fan-out to the WebSocket hub feeding the React dashboard.

6.1 ASCII architecture diagram





6.2 Data flow

- 1. Client POSTs a task → API inserts row in Postgres (status=queued) and ZADD-s into queue:{type}.
- 2. Worker runs Lua ZPOPMIN + HSET to atomically lease the task into inflight:{worker}.
- 3. Worker updates Postgres (status=running, started_at, worker_id) and executes the handler.
- 4. On success → ACK (HDEL inflight, update Postgres → succeeded, publish event).
- 5. On error and attempts < max → NACK with backoff (delayed ZSET); else push to DLQ.
- 6. Coordinator tick: promote due delayed entries; scan worker registry for missing heartbeats; reclaim in-flight tasks.
- 7. Every transition publishes a task.Event on events:tasks → WebSocket hub → dashboard.

6.3 Scaling characteristics

Component	Scaling strategy
API server	Stateless; scale horizontally behind any L4/L7 load balancer.
Workers	Stateless; add replicas for more throughput. Per-worker concurrency tunable via WORKER_CONCURRE
Coordinator	Run as a singleton (or leader-elected). Work is O(workers + due_delayed) per tick.
Redis	Single instance for simplicity; can be sharded by task type since keys are namespaced by type.
PostgreSQL	Vertical scale + read replicas for the dashboard. Writes are bounded by task throughput.
Dashboard	Static SPA; trivially CDN-cacheable. Live data over WebSocket.

7. Low-Level Design

7.1 Redis key layout

Key	Type	Purpose
queue:{type}	ZSET	Ready tasks. Score = priority*1e13 + enqueueMs.
delayed:{type}	ZSET	Retry/delay queue. Score = available_at epoch ms.
inflight:{worker}	HASH	Tasks currently leased by a worker (task_id → envelope JSON).

Key	Type	Purpose
hb:{worker}	STRING	Heartbeat with TTL (default 9s, refreshed every 3s).
dlq:{type}	LIST	Dead-letter envelopes for tasks past max_attempts.
events:tasks	PUB/SUB	Task lifecycle events fanned out to WebSocket clients.

7.2 PostgreSQL schema (essentials)

```
-- tasks: canonical record
CREATE TABLE tasks (
  id          UUID PRIMARY KEY,
  type        TEXT NOT NULL,
  payload     JSONB NOT NULL,
  priority    INT NOT NULL,
  status      TEXT NOT NULL,
  attempts    INT NOT NULL DEFAULT 0,
  max_attempts INT NOT NULL,
  available_at TIMESTAMPTZ NOT NULL,
  started_at  TIMESTAMPTZ,
  finished_at TIMESTAMPTZ,
  worker_id   TEXT,
  result      JSONB,
  last_error  TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- task_events: immutable audit trail
CREATE TABLE task_events (
  id          BIGSERIAL PRIMARY KEY,
  task_id     UUID NOT NULL REFERENCES tasks(id),
  status      TEXT NOT NULL,
  worker_id   TEXT,
  message     TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- workers: registry + last heartbeat
CREATE TABLE workers (
  id          TEXT PRIMARY KEY,
  hostname    TEXT,
  last_heartbeat TIMESTAMPTZ NOT NULL
);
```

7.3 Task envelope and domain model

Persisted record (Go struct):

```
type Task struct {
  ID          uuid.UUID
  Type        string
  Payload     json.RawMessage
  Priority     int
  Status      Status          // queued|running|succeeded|failed|retrying|dead|canceled
  Attempts    int
  MaxAttempts int
  AvailableAt time.Time
  StartedAt   *time.Time
  FinishedAt  *time.Time
  WorkerID    *string
  Result      json.RawMessage
  LastError   string
}
```



```

    CreatedAt    time.Time
    UpdatedAt    time.Time
}

```

7.4 Atomic lease (Lua sketch)

```

-- KEYS[1] = queue:{type}
-- KEYS[2] = inflight:{worker}
-- ARGV[1] = nowMs
local popped = redis.call('ZPOPMIN', KEYS[1], 1)
if #popped == 0 then return nil end
local envelope = popped[1] -- JSON-encoded task
local taskId = cJSON.decode(envelope).id
redis.call('HSET', KEYS[2], taskId, envelope)
return envelope

```

Because the script runs server-side in a single Redis command, the pop and the in-flight registration are atomic – the task cannot be lost between the two operations.

7.5 Worker loop

- Register self in workers table and start heartbeat goroutine.
- Spawn `WORKER_CONCURRENCY` goroutines, each calling Lease → Execute → Ack/Nack in a loop.
- On panic or context cancellation, NACK the current task (so the lease is not held until coordinator reclaim).
- On graceful shutdown, drain in-flight, stop heartbeat, deregister.

7.6 Coordinator loop

- **Promote delayed:** `ZRANGEBYSCORE delayed:{type} -inf now`; for each due entry `ZADD` to ready queue and `ZREM` from delayed.
- **Reclaim dead workers:** for every worker in the registry, check `EXISTS hb:{worker}`; if missing, `HGETALL inflight:{worker}`, re-enqueue, `DEL` hash, mark Postgres rows back to queued/retrying.
- **Tick interval:** `COORDINATOR_SWEEP_INTERVAL` (default 1s).

7.7 WebSocket Hub

- One Redis `SUBSCRIBE` goroutine reads from `events:tasks`.
- A registry of client connections is protected by a mutex; broadcasts are non-blocking (slow clients are dropped).
- Each connection has a ping/pong loop to detect dead sockets.

7.8 Retry/backoff formula

```

delay = min(RETRY_MAX_DELAY,
            RETRY_BASE_DELAY * 2^(attempt-1))
delay = uniform_random(0, delay) // full jitter
score = nowMs + delayMs
ZADD delayed:{type} score envelope

```

8. Configuration Reference

Variable	Default	Purpose
DATABASE_URL	host=localhost port=5432 user=postgres dbname=taskq sslmode=disable	PostgreSQL DSN
PGPASSWORD	(libpq fallback)	Password if not in DSN
REDIS_URL	redis://localhost:6379/0	Redis connection URL
HTTP_ADDR	:8080	API bind address
WORKER_CONCURRENCY	4	Goroutines per worker process
HEARTBEAT_INTERVAL	3s	Cadence of hb:{worker} refresh
HEARTBEAT_TTL	9s	TTL on hb:{worker} (death threshold)
COORDINATOR_SWEEP_INTERVAL	1s	Promotion + reclaim tick
RETRY_BASE_DELAY	500ms	Initial backoff
RETRY_MAX_DELAY	60s	Cap for exponential backoff
LEASE_TTL	60s	Soft lease duration
TASK_TYPES	(all)	CSV of types this worker accepts

Built-in handlers

Type	Payload	Behaviour
echo	any	Returns the payload unchanged.
sleep	{ seconds: float }	Sleeps the given duration then returns.
flaky	{ fail_rate: 0..1 }	Fails probabilistically; exercises retries.
email	{ to, subject }	Validates and logs a 'sent' event.

9. Testing & Validation

9.1 Unit tests

Run all Go unit tests:

```
go test ./...
```

Coverage highlights:

- Redis queue layer (priority ordering, lease/ack, delayed promotion, in-flight reclaim, DLQ, heartbeat TTL) using miniredis.
- Exponential-backoff calculator (monotonicity, jitter bounds, cap).
- Handler registry (registration, lookup, types listing).

9.2 Load test

```
# Submit 10,000 tasks at concurrency 64 of type flaky
./scripts/loadtest.sh 10000 64 flaky
# Expected output ends with:
```

```
# FINAL by_status=... dead=0 failure_rate=0.0000%
```

9.3 Crash-recovery test

```
# While loadtest is running, in another terminal:  
./scripts/kill-worker.sh  
# The coordinator detects the dead worker within ~5s,  
# requeues its in-flight tasks, and they complete on other workers.
```

9.4 DLQ test

```
curl -X POST http://localhost:8080/api/tasks \  
  -H 'Content-Type: application/json' \  
  -d '{"type":"flaky","payload":{"fail_rate":1.0},"priority":5,"max_attempts":2}'  
# The task retries then moves to the DLQ. View / redrive in the dashboard.
```

End of document.